DD MM YYYY
25 10 2025

Module - 01

01) What is object oriented programming? With a sample program, explain the general format of a C++ program.

->> The word object-oriented is the combination of two words object and oriented.

> The object-oriented programming is basically programming design methodology that models software design and the ~~functions~~ to data or objects rather than functions and logic.

General Form of C++ program

Although individual styles will differ, most C++ programs will have this general form.

```
#include
base-class declarations
derived class declarations
nonmember function prototypes
int main()
{
  //....
}
nonmember function definations
```

In most some projects, the class declarations will be put into a header file and included with each module. But the general organization of a program remains the same.

D D M M Y Y Y Y

02) What is Friend function, Friend classes and Inline function. Explain with an example.

→ ~~Data hiding is a fundamental~~

* Friend function

> Data hiding is a fundamental concept of object-oriented programming. It restricts the access of private members from outside of the class.

> Similarly, protected members can only be accessed by derived classes and are inaccessible from outside. For example,

```
class My class {
        private: int Member 1;
                }
int Main()
        {
        Myclass obj; // Error! cannot access private Members
        obj.Member1 = 5;
        }
```

> A friend function can access the private and protected data of a class. We declare a friend function using the friend keyword inside the body of the class.

```
class className {
        ....
        friend returntype functionName (arguments);
        ....
        }
```

Ex:

```
# include<iostream>
using namespace std;
class demo
{
```

```
private:
int a=5;
friend void display (demo);
};
void display (demo d)
{
cout<<"a: "<<d.a<<endl;
}
int main ()
{
demo d;
display (d);
return 0;
}
```

Friend classes

→ we can also use a friend classes in c++ using the friend keyword. For example,

```
class classB;
class classA{
        friend class classB;
        }
class classB {
        - - - - - -
        }
```

Ex:

```
#include<iostream>
using namespace std;
class test1
{
```

```
int a,b;
  public:
  void get()
  {
   cout<<"Enter 2 integers";
   cin>>a>>b;
  }
  friend class test2;
};
class test1
{
  test1 t1;
  public:
  void put()
  {
   t1.get();
   cout<<"a:"<<t1.a<<endl;
   cout<<"b:"<<t1.b<<endl;
  }
};
int main()
{
  test2 t2;
   t2.put();
}
```

D D M M Y Y Y Y

Inline function

→ In c++, we can declare a function as inline. This copies the function to the location of the function call in compile-time and may make the program execution faster.

→ To create an inline function, we use the inline keyword.
inline returntype functionName (parameters) { // code }
Notice the use of keyword inline before the function definition.

Ex:

```
#include<iostream>
using namespace std;
inline void displayNum(int num)
{ cout<<num<<endl; }
int main() {
    displayNum(5);
    displayNum(8);
    displayNum(666);
    return 0;
}
```

output: 5 8 666

03) What is constructor and destructors? Explain the scope resolution with an syntax.

→ A constructor is a particular member function having the same class name. It calls automatically whenever the object of the class is created.

→ A Destructor have the same class name preceded by (~) tilde symbol. It removes and destroys the memory of the object, which the constructor allocated during the

creation of an object.

Syntax: The syntax of the constructor in c++ are given below.

```
class class_name
{
....
public
class_name([parameter List])
   {
     ....
   }
};
```

Syntax: The syntax of the destructor in c++ are given below.

```
class class_name
{
  ....;
  ....;
public:
  xyz();  // constructor
 ~xyz();  // destructor
};
```

The scope resolution in c++

→ It is used to access the global variables or member functions of a program when there is a local variable or member function with the same name.

→ It is used to define the member function outside of a class.

D D M M Y Y Y Y

> The operator is represented as the double(::) symbol.

Ex:

```
#include<iostream>
using namespace std;
class rectangle
{
  private:
    int length;
    int breadth;
  public:
  rectangle (int l, int b)
      {
          length = l;
          breadth = b;
      }
    int area()
    {
      return length * breadth;
    }
int perimeter ();
};
int rectangle::perimeter()
{
  return 2 * (length + breadth);
}
int main()
{
  rectangle r(8,3);
  cout << "Area is: " << r.area() << endl;
  cout << "perimeter is: " << r.perimeter();
}
```

output:

```
Area is : 24
Perimeter is : 22
```

04) How objects are passed to the functions, explain.

--> In c++ programming, we can pass objects to a function in a similar manner as passing regular arguments.

syntax: function_name(object_name)

Ex:

```
#include<iostream>
using namespace std;
class A {
    public:
        int age=20;
        char ch='A';
    void display(A &a) {
        cout<<"Age="<<a.age<<endl;
        cout<<"Character="a.ch<<endl;
    }
};
int main()
{
    A obj;
    obj.display(obj);
    return 0;
}
```

o/p

```
Age=20
Character=A
```

D D M M Y Y Y Y

Pass by value : This creates a shallow local copy of the object in the function scope. Things you modify here won't be reflected in the object passed to it.

Pass by reference : The passes a reference to the object to the function. Things you modify here will be reflected in the object passed to it. No copy of the object is created.

05) List out difference b/w. Procedural oriented programming and object oriented programming.

Procedural-oriented Programming	object-oriented Programming
> It is knows as POP.	> It is known as OOP.
> Procedural Programming languages are not as faster as object-oriented.	> The object-oriented programming languages are faster and more effective.
> Procedural uses procedures, Modules, procedure calls.	> object-oriented uses objects, classes, messages.
> Top-down approach.	> Bottom-up approach.
> Access Modifiers are not supported.	> Access Modifiers are supported.
> Functions are preferred over data.	> security and accessebility are preferred.
> If the size of the problem is small, POP is preferred.	> If the size of the problem is big, OOP is preferred.
> C++, C#, Java, Python, etc. are the example of OOP languages.	> C, BASIC, COBOL, Pascal etc. are the examples POP languages.

06) Explain basic concepts of OOPS.

→ i) class

- The building block of c++ that leads to object-oriented programming is a class.
- It is user-defined data type, which holds its own data members and member functions, which can be accessed and used by creating an instance of that class. A class is like a blueprint for an object.
- For example: consider the class of cars. There may be many cars with different names and brandes but all of them will have 4 wheels, speed limits, Milage range etc. so here, the car is the class, and wheels, speed limits and Mileage are their properties.

ii) Object

- An object is an identifiable entity with some characteristics and behaviour. An object is an instance of a class.
- When a class is defined, no memory is allocated but when it is instantiated memory is allocated.

iii) Encapsulation

- It is defined as wrapping up data and information under a single unit.
- In object-oriented Programming, Encapsulation is defined as binding together the data and the functions that manipulate them.

D D M M Y Y Y Y

iv) Abstraction

→ It means displaying only essential information and hiding the details.

→ Data abstraction refers to providing only essential information about the data to the outside world, hiding the background details or implementation.

v) Polymorphism

→ The word polymorphism means having many forms. In simple words, we can define polymorphism as the ability of a message to be displayed in more than one form. A person at the same time can have different characteristics

→ Operator overloading: The process of making an operator exhibit different behaviours in different instances is knows as operator overloading.

→ Function overloading: Function overloading is using a single function name to perform different types of tasks. Polymorphism is extensively used in implementing inheritance.

vi) Inheritance

→ The capability of a class to derive properties and characteristics from another class is called Inheritance.

- Sub class: The class that inherits properties from another class is called sub class.
- Super class: The class whose properties are inherited by a sub-class is called superclass.
- Reusability: Inheritance supports the concept of reusability when we want to create a new class and there is already a class that includes some of the code that we want, we

can derive our new class from the existing class.
EX: Cat, Dog, Cow can be derived class of Animal Base class.

vii) Dynamic Binding
→ In this the code to be executed in response to the function call is decided at runtime.
→ C++ has virtual functions to support this. Because dynamic binding is flexible, it avoids the drawbacks of static binding, which connected the function call and definition at build time.

viii) Message passing
→ Objects communicate with one another by sending and receiving information.
→ A message for an object is a request for the execution of a procedure and therefore will invoke function in the receiving object that generates the desired results.

07) Explain the concept of static data members and static member functions with an ex.
→ Static data members
→ These are class members that are declared using static keywords.
→ It is initiated before any object of this class is created, even before the main starts.
→ It is visible only within the class, but its lifetime is the entire program.
syntax:
static data_type data_member_name;

DD MM YYYY

Ex:

```
#include<iostream>
using namespace std;
class circle
{
  private:
    float radius;
    static float pi;
  public:
    circle(float radius)
    {
      radius = radius;
    }
    float Area()
    {
  return pi * radius *
  radius;
    }
};
  float circle::pi = 3.14f;
  int main()
  {
    circle c1(4);
  cout << "Area = " << c1.Area();
    return 0;
  }
```

o/p

Area = 50.24

D D M M Y Y Y Y

Static Member Functions

- It is a member function of a class that can be called without creating an object of the class.
- It is defined as using the "static" keyword, and it is typically declared within the class definition but outside of any member function.

syntax:

```
class className {
    static return_type function_Name (parameter list);
};
```

Ex:

```
#include<iostream>
using namespace std;
class Test
{
  public:
    static void display();
};
void Test::Display()
{
  cout<<"This is a static member function";
}
int main()
{
  Test::Display();
  return 0;
}
```

op

This is a static member function